CLAUDE CODE SOURCE STUDY · VOL. 06

확장, 제품 변형, 운영과 구조 진화

MCP·plugin·skill, remote/bridge transport, build/runtime gate, configuration, migration, telemetry를 하나의 변화 관리 문제로 읽는다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 새 capability와 remote surface가 어떻게 발견·정규화·권한화·전송·관찰되고, 어떤 불변조건 아래 구조를 개선해야 하는지 설명할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

MCP, plugin, skill은 서로 다른 확장 surface다. remote/bridge는 local message와 authority를 wire protocol로 다시 투영한다. build-time feature와 runtime configuration/GrowthBook이 제품 variant를 만든다.

구현에서 직접 확인되는 범위

외부 identity는 normalization/schema를 거쳐 Tool 언어로 투영되고, remote SDK adapter와 bounded reconnect가 local runtime과 transport failure를 연결한다. settings/migration/analytics가 장기 운영을 보조한다.

운영을 말할 때 남겨야 할 경계

코드에는 reconnect, polling, batching, checksum, migration이 보이지만 실제 server replay·조직 운영·활성 값은 unknown이다.

이 보고서가 추측으로 채우지 않는 것

plugin 공급망 정책 전체, remote server의 replay/retention contract, remote managed settings 실제 배포 범위, telemetry retention/backend, 그리고 공식 refactor roadmap은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

효과·위임

MCP `remote capability adapter`

protocol/trust

"protocol/trust"가 확인되어야 다음의 "Plugin" 영역이 "supply chain"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

효과·위임

Plugin `code/config installation`

supply chain

"supply chain"가 확인되어야 다음의 "Skill" 영역이 "behavior provenance"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

효과·위임

Skill `instruction dependency`

behavior provenance

"behavior provenance"가 확인되어야 다음의 "Remote/bridge" 영역이 "connection and authority"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

관찰·정산

Remote/bridge `wire↔local projection`

connection and authority

"connection and authority"가 확인되어야 다음의 "Feature" 영역이 "variant"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

진입 경계

Feature `build/runtime variability`

variant

"variant"가 확인되어야 다음의 "Settings/migration" 영역이 "compatibility"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

기록·복구

Settings/migration `durable config evolution`

compatibility

"compatibility"가 확인되어야 다음의 "Telemetry" 영역이 "operational model"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

관찰·정산

Telemetry `events→sink`

operational model

이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 확장은 서로 다른 trust boundary다

MCP, plugin, skill을 “확장 기능” 하나로 묶으면 effect와 provenance 차이가 사라진다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	효과·위임 MCP server name normalization	사례	효과·위임 MCP capability가 Tool이 되기까지
구분	효과·위임 MCP·Plugin·Skill 비교	알고리즘	효과·위임 Background plugin installation queue
해석	효과·위임 Skill은 코드보다 가볍지만 authority와 무관하지 않다		

코드 독해의 시작점

`src/services/mcp/**` → `src/services/plugins/**` → `src/skills/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A32 · 독자의 질문

MCP server name normalization 은 어떤 시스템 문제를 닫는가

외부 MCP server name에는 공백과 점이 있지만 tool namespace는 제한된 pattern을 요구한다. 순수 normalization은 interop를 돕지만 서로 다른 원본을 같은 문자열로 만들 수 있다.

코드가 필요해지는 이유

공백·점 등이 있는 server name을 tool namespace pattern에 맞춘다.

OBSERVED normalization 함수가 원래 server name을 deterministic normalized string으로 투영한다.

INFERENCE registry가 original identity와 collision을 소유하고 pure adapter는 표현 변환만 소유한다.

이 절에서 답할 결정 질문

canonical name은 wire 호환을 위해 어디까지 쓰고 왜 authority identity로 승격하면 안 되는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **canonicalization과 namespace collision.** 여러 표기를 하나로 만들면 비교는 쉬워지지만 many-to-one 변환이면 원본 identity를 복원할 수 없다.
>
> 1–23행에서 치환·길이 규칙을 보고 서로 다른 입력이 같은 출력이 되는 반례를 만든다.

이 설명을 확인할 코드 src/services/mcp/normalization.ts:1-23

ALGORITHM A32 · 실제 코드

OBSERVED

MCP server name normalization 의 결정점은 어디에 있는가

외부 이름을 API-safe identity로 바꾸되 delimiter 충돌을 줄인다.

입력 표현. 원래 server name

보존되는 작업 상태. normalized string

pure normalization function

src/services/mcp/normalization.ts:1-23

```
1     /**
2      * Pure utility functions for MCP name normalization.
3      * This file has no dependencies to avoid circular imports.
4      */
5
6     // Claude.ai server names are prefixed with this string
7     const CLAUDEAI_SERVER_PREFIX = 'claude.ai '
8
9     /**
10     * Normalize server names to be compatible with the API pattern ^[a-zA-Z0-9_-]{1,64}$
11     * Replaces any invalid characters (including dots and spaces) with underscores.
12     *
13     * For claude.ai servers (names starting with "claude.ai "), also collapses
14     * consecutive underscores and strips leading/trailing underscores to prevent
… 6 lines omitted; use the source link for the complete range …
21      }
22      return normalized
23    }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A32 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

원래 server name은 처음부터 신뢰된 실행 상태가 아니다. normalized string가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 허용 문자 외 문자를 underscore로 바꾼다. 이 결과가 다음 판단 “claude.ai prefix 이름은 연속 underscore를 접는다.”의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** claude.ai prefix 이름은 연속 underscore를 접는다. 이 결과가 다음 판단 “leading/trailing underscore를 제거한다.”의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** leading/trailing underscore를 제거한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

결과는 API name pattern을 만족한다. '__' delimiter와의 간섭을 줄인다. display/original identity는 별도로 보존해야 한다.

ALGORITHM A32 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 결과는 API name pattern을 만족한다.
- **프로토콜·식별의 질문으로 읽기.** '__' delimiter와의 간섭을 줄인다.
- **상태·계산 모델의 질문으로 읽기.** display/original identity는 별도로 보존해야 한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 서로 다른 원본이 같은 normalized name으로 충돌
- 64자 제한 별도 처리 필요
- normalized identity를 authority로 오해

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED protocol interop를 위해 pure deterministic adapter를 둔다.

RECOMMENDATION registry에서 original+normalized pair와 collision check를 유지한다.

ALGORITHM A32 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

server name을 허용 문자 집합의 namespace string으로 바꾼다.

실패 뒤 남는 상태

공백·점 변형과 길이 제한은 다른 server를 같은 이름으로 합칠 수 있다.

OBSERVED	작고 순수한 deterministic normalization 함수가 있다.
CONDITIONAL	registry가 collision을 별도로 거부하거나 disambiguate할 때 interop adapter로 안전하다.
UNKNOWN	실제 연결 server name 분포와 collision 빈도는 알 수 없다.
NEXT READ	A20 coarse permission namespace와 같은 normalization을 공유하는지 확인해야 한다.

다시 적용할 질문

1. a.b와 a b가 같은 output이면 registry와 permission rule은 어떻게 구분해야 하는가?
2. 64자 truncation을 추가하면 기존 stored identity migration이 왜 필요한가?

SELECTIVE EXECUTION TRACE

OBSERVED

MCP capability가 Tool이 되기까지

외부 schema가 local authority를 자동 획득하지 않는다.

책임 연결

Discover

server capability와 schema를 읽는다.

이때 남은 상태와 판단이 “Normalize”에서 사용할 전제가 된다.

책임 연결

Normalize

server/tool identity를 local namespace로 바꾼다.

이때 남은 상태와 판단이 “Adapt”에서 사용할 전제가 된다.

책임 연결

Adapt

input/result를 Tool contract로 감싼다.

이때 남은 상태와 판단이 “Register”에서 사용할 전제가 된다.

책임 연결

Register

현재 tool pool과 model schema에 노출한다.

이때 남은 상태와 판단이 “Authorize”에서 사용할 전제가 된다.

판단 경계

Authorize

local permission과 transport trust를 적용한다.

이때 남은 상태와 판단이 “Project”에서 사용할 전제가 된다.

책임 연결

Project

MCP metadata와 content를 SDK/UI message로 바꾼다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

외부 protocol validity와 local 실행 권한은 별도다.

어디에서 설명이 틀어지는가

이름 충돌이나 server reconnect 시 stale Tool이 남을 수 있다.

이 설명을 확인할 코드 src/services/mcp/normalization.ts:1-23 src/services/tools/toolExecution.ts:283-335

DISTINCTION TABLE

OBSERVED

MCP·Plugin·Skill 비교

확장 단위마다 설치·실행·신뢰 대상이 다르다.

종류	추가하는 것	핵심 위험
MCP	remote tools/resources	server identity·transport·effect
Plugin	code/config/hooks/skills	설치 provenance·update
Skill	model instruction/context	instruction precedence·behavior drift

이 표를 읽는 관점

같은 enable/disable UI가 있어도 내부 control은 동일하지 않다.

ALGORITHM A33 · 독자의 질문

Background plugin installation queue은 어떤 시스템 문제를 닫는가

startup에 plugin 설치를 기다리면 느리고 background로 돌리면 UI가 extension을 아직 쓸 수 없는 중간 상태를 가진다. marketplace 갱신과 부분 설치도 별도 상태다.

코드가 필요해지는 이유

시작 시 필요한 plugin을 설치·갱신하고 진행 상태를 AppState에 반영한다.

OBSERVED — PluginInstallationManager가 marketplace status와 진행/성공/오류 state를 AppState에 projection한다.

INFERENCE — manager가 background job lifetime을 소유하고 UI/registry는 projection된 availability를 소비한다.

이 절에서 답할 결정 질문

diff→pending→progress→settled 흐름에서 availability와 provenance를 어떻게 함께 표현하는가?

이 설명을 확인할 코드 src/services/plugins/PluginInstallationManager.ts:60-120

ALGORITHM A33 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/plugins/PluginInstallationManager.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

background job과 eventual consistency

요청 직후 UI state와 filesystem installed state가 잠시 다를 수 있다. 완료 여부뿐 아니라 어느 manifest와 marketplace revision을 적용했는지 필요하다.

> **실제 코드로 돌아가면.** 60–120행에서 diff 계산, pending state, progress reconcile이 어떤 identity로 연결되는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`background job · eventual consistency`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

권한·신뢰 경계
`supply-chain provenance`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

ALGORITHM A33 · 실제 코드

OBSERVED

Background plugin installation queue의 결정점은 어디에 있는가

plugin install은 UI blocking을 피하면서 marketplace 상태와 실패를 조정한다.

입력 표현. plugin/marketplace state, 그리고 installation requests

보존되는 작업 상태. marketplace status, 그리고 진행/성공/오류

diff→pending state→reconcile progress

src/services/plugins/PluginInstallationManager.ts:60-120

```
60    export async function performBackgroundPluginInstallations(
61      setAppState: SetAppState,
62    ): Promise<void> {
63      logForDebugging('performBackgroundPluginInstallations called')
64
65      try {
66        // Compute diff upfront for initial UI status (pending spinners)
67        const declared = getDeclaredMarketplaces()
68        const materialized = await loadKnownMarketplacesConfig().catch(() => ({}))
69        const diff = diffMarketplaces(declared, materialized)
70
71        const pendingNames = [
72          ...diff.missing,
73          ...diff.sourceChanged.map(c => c.name),
… 44 lines omitted; use the source link for the complete range …
118           }
119         },
120       })
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A33 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

plugin/marketplace state, 그리고 installation requests은 처음부터 신뢰된 실행 상태가 아니다. marketplace status, 그리고 진행/성공/오류가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 대상 marketplace 상태를 갱신한다. 이 결과가 다음 판단 "필요한 install/update를 비동기로 실행한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 필요한 install/update를 비동기로 실행한다. 이 결과가 다음 판단 "결과를 state와 notification에 반영한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 결과를 state와 notification에 반영한다. 이 결과가 다음 판단 "실패를 전체 app startup과 분리한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 실패를 전체 app startup과 분리한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다. 설치 source와 version을 유지한다. 새 executable capability는 다음 tool reload 전에 완결된다.

ALGORITHM A33 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** 한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** 설치 source와 version을 유지한다.
- **동시성·시간·합의의 질문으로 읽기.** 새 executable capability는 다음 tool reload 전에 완결된다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 부분 설치
- 중복 background operation
- marketplace update 중 provenance drift

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED interactive startup latency와 extension availability를 절충한다.

RECOMMENDATION transactional staging과 signed manifest/receipt를 명시한다.

ALGORITHM A33 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

필요한 install/update diff를 queue에 넣고 background 실행 결과를 AppState에 반영한다.

effect가 시작되고 관찰되는 경계

network fetch와 filesystem/plugin state 변경이 발생한다.

현재 설계가 선택한 trade-off와 비용

interactive startup latency와 extension availability를 절충한다. 이 선택의 계산·운영 비용은 plugin 수와 network/disk install 비용.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

부분 설치, 중복 job, marketplace revision drift는 UI success와 실제 provenance를 갈라놓는다. 이때 함께 대입할 반례는 부분 설치, 중복 background operation, 그리고 marketplace update 중 provenance drift이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

request id, manifest digest, marketplace revision, staged/committed disposition을 기록해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A33 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	installation request diff와 pending/progress 상태 projection이 있다.
CONDITIONAL	job이 단일화되고 같은 marketplace snapshot을 사용할 때 UI와 installed state가 수렴한다.
UNKNOWN	package signature 검증과 실제 transactional staging 보장은 확인되지 않는다.
NEXT READ	signed manifest와 staging receipt를 도입하기 전 install filesystem effect를 inventory해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 설치 중 marketplace가 갱신되면 현재 job은 어느 revision을 계속 사용해야 하는가?
2. 일부 파일만 설치된 뒤 crash하면 다음 startup diff가 그 상태를 어떻게 분류하는가?

이 설명을 확인할 코드 src/services/plugins/PluginInstallationManager.ts:60-120

CONCEPT IN CONTEXT

INFERENCE

Skill은 코드보다 가볍지 만 authority와 무관하지 않다

instruction dependency는 실행 파일이 아니어도 model이 어떤 도구를 어떻게 쓰는지 바꾼다.

코드에서 읽어낼 수 있는 단서

출발 단서	skill content는 context에 주입된다.
연결되는 단서	동적 discovery와 allowedTools가 turn state에 영향을 준다.
연결되는 단서	plugin이 skill을 공급할 수 있다.

이 단서들을 하나의 설계로 묶으면

instruction provenance와 precedence를 잃으면 행동 변화의 원인을 설명하기 어렵다.

시스템 이해에 남는 결론

보고서는 skill을 문서 파일로만 보지 말고 behavior dependency로 다뤄야 한다.

이 설명을 확인할 코드 src/screens/REPL.tsx:2701-2726

02

Remote와 bridge는 local 의미를 transport protocol로 다시 투영한다

remote control을 “같은 REPL의 네트워크 버전”으로 단순화하지 않는다. WebSocket lifecycle, SDK message adapter, inbound command gate, permission callback이 local state와 별도 failure domain을 만든다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 WebSocket close code별 bounded reconnect	코드	책임 연결 SDK message adapter는 wire representation의 정보 손실을 명시한다
코드	진입 경계 REPL bridge는 remote input을 command queue에 넣기 전에 identity와 command authority를 제한한다	구분	의미 연결 Remote surface의 네 protocol을 분리한다
사례	효과·위임 remote prompt가 local effect를 시작하는 경계		

코드 독해의 시작점

`src/remote/SessionsWebSocket.ts` → `src/remote/sdkMessageAdapter.ts` → `src/hooks/useReplBridge.tsx` → `src/bridge/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A44 · 독자의 질문

WebSocket close code별 bounded reconnect은 어떤 시스템 문제를 닫는가

remote session의 network는 일시적으로 끊길 수 있지만 server가 session 없음이나 terminal close를 확정했다면 무한 reconnect는 잘못이다. close code와 attempt budget이 다음 상태를 결정한다.

코드가 필요해지는 이유

remote session의 일시적 network 단절을 복구하면서 server가 확정한 종료나 존재하지 않는 session을 무한 재시도하지 않는다.

OBSERVED SessionsWebSocket이 ws handle, connection state, reconnect attempts, session-not-found retries, timer를 소유한다.

INFERENCE transport adapter가 socket lifetime과 재접속을 소유하고 상위 session은 remote event의 제품 의미를 소유한다.

이 절에서 답할 결정 질문

manual close, transient close, session-not-found, token refresh를 어떤 generation과 budget으로 구분하는가?

이 설명을 확인할 코드 src/remote/SessionsWebSocket.ts:208-300 src/remote/RemoteSessionManager.ts:106-183

ALGORITHM A44 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/remote/SessionsWebSocket.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

connection state machine과 bounded retry

open/closing/closed/reconnecting은 boolean 하나로 표현하기 어렵다. close reason마다 terminal 여부와 별도 retry budget을 두어 availability와 retry storm을 절충한다.

> **실제 코드로 돌아가면.** 208–300행에서 close code 분기, counter 증가/reset, timer 설치와 manual reconnect를 함께 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`state machine`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

구성·변형·성능
`anti-corruption`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

동시성·시간·합의
`retry budget`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

ALGORITHM A44 · 최소 예제

늦은 reconnect timer가 새 connection generation을 덮지 않아야 한다

첫 socket의 timer가 예약된 뒤 사용자가 수동 reconnect하는 interleaving이다.

설명을 위한 최소 실행 예제

```
socket1 closes -> timer1 scheduled
manualReconnect() -> socket2 opens
timer1 fires -> must not replace socket2
// generation1 timer는 generation2 state를 쓸 권한이 없음
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
close 1	transient로 timer1 예약	generation1의 복구 작업이다.
manual reconnect	socket2 연결	현재 owner가 generation2로 바뀐다.
timer1 fire	stale callback	generation check 없으면 healthy socket2를 교체한다.

예제를 실제 구현에 다시 연결한다

timer callback이 현재 ws/state를 검증하는지와 counter reset 시점을 실제 코드에서 본다.

ALGORITHM A44 · 실제 코드

OBSERVED

WebSocket close code별 bounded reconnect의 결정점은 어디에 있는가

연결 종료를 permanent, session-not-found transient, ordinary transient로 분류하고 서로 다른 retry budget으로 terminal 여부를 결정한다.

입력 표현. 현재 WebSocketState, close code, reconnect attempt counters, 그리고 retry constants

보존되는 작업 상태. ws handle, state, reconnectAttempts, sessionNotFoundRetries, 그리고 reconnectTimer

message validation과 reconnect state machine

src/remote/SessionsWebSocket.ts:208-300

```
208      * Handle incoming WebSocket message
209      */
210     private handleMessage(data: string): void {
211       try {
212         const message: unknown = jsonParse(data)
213
214         // Forward SDK messages to callback
215         if (isSessionsMessage(message)) {
216           this.callbacks.onMessage(message)
217         } else {
218           logForDebugging(
219             `[SessionsWebSocket] Ignoring message type: ${typeof message === 'object' && message !== null
&& 'type' in message ? String(message.type) : 'unknown'}`,
220           )
221         }
… 76 lines omitted; use the source link for the complete range …
298       }, delay)
299     }
300
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A44 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

현재 WebSocketState, close code, reconnect attempt counters, 그리고 retry constants은 처음부터 신뢰된 실행 상태가 아니다. ws handle, state, reconnectAttempts, sessionNotFoundRetries, 그리고 reconnectTimer가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 수신 JSON을 parse하고 SessionsMessage runtime guard를 통과한 값만 callback에 전달한다. 이 결과가 다음 판단 "close에서 ping을 멈추고 기존 state를 보존한 뒤 handle을 해제한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** close에서 ping을 멈추고 기존 state를 보존한 뒤 handle을 해제한다. 이 결과가 다음 판단 "permanent close code면 즉시 terminal callback을 호출한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** permanent close code면 즉시 terminal callback을 호출한다. 이 결과가 다음 판단 "4001이면 compaction 중 transient 가능성을 인정하되 전용 retry budget과 증가 delay를 적용한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 4001이면 compaction 중 transient 가능성을 인정하되 전용 retry budget과 증가 delay를 적용한다. 이 결과가 다음 판단 "일반 close는 이전에 connected였고 budget이 남을 때만 reconnect를 예약한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** 일반 close는 이전에 connected였고 budget이 남을 때만 reconnect를 예약한다. 이 결과가 다음 판단 "timer가 다시 fresh token으로 connect를 시도한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** timer가 다시 fresh token으로 connect를 시도한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

permanent close는 reconnect하지 않는다. 동시에 하나의 reconnect timer만 다음 연결을 소유한다. 처음부터 연결되지 못한 상태와 연결 뒤 끊긴 상태를 같은 retry로 처리하지 않는다. unknown/invalid message는 application callback으로 승격하지 않는다.

ALGORITHM A44 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** permanent close는 reconnect하지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** 동시에 하나의 reconnect timer만 다음 연결을 소유한다.
- **구성·변형·성능의 질문으로 읽기.** 처음부터 연결되지 못한 상태와 연결 뒤 끊긴 상태를 같은 retry로 처리하지 않는다.
- **상태·계산 모델의 질문으로 읽기.** unknown/invalid message는 application callback으로 승격하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- close와 manual reconnect timer race
- 4001 의미가 server version에서 달라짐
- token refresh와 reconnect ordering
- 연결 복구 뒤 event 중복 또는 gap

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED availability를 높이되 server terminal semantics와 retry storm 방지를 우선하는 explicit connection state machine을 둔다.

RECOMMENDATION connection generation과 last received event identity를 handshake에 포함하고 reconnect 뒤 replay/duplication disposition을 명시한다.

ALGORITHM A44 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

message를 validate하고 close code별 terminal/retry 결정을 내린 뒤 bounded delay로 새 socket을 만든다.

effect가 시작되고 관찰되는 경계

network connection과 event replay가 재시작되며 중복 또는 gap 가능성이 생긴다.

현재 설계가 선택한 trade-off와 비용

availability를 높이되 server terminal semantics와 retry storm 방지를 우선하는 explicit connection state machine을 둔다. 이 선택의 계산·운영 비용은 message 처리 O(payload parse), reconnect 횟수는 두 bounded counter에 제한된다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

timer race, server close-code 의미 drift, token ordering, replay disposition 부재는 연결은 복구돼도 session state를 틀리게 한다. 이때 함께 대입할 반례는 close와 manual reconnect timer race, 4001 의미가 server version에서 달라짐, token refresh와 reconnect ordering, 그리고 연결 복구 뒤 event 중복 또는 gap이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

connection generation, close code, attempt budget, last event id, replay outcome을 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A44 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	명시적 WebSocketState, close-code 분기, retry counters와 timer가 있다.
CONDITIONAL	server와 client가 close semantics를 공유하고 stale timer가 current generation을 검사할 때 bounded reconnect가 안전하다.
UNKNOWN	server version별 4001 의미와 reconnect 이후 replay/duplication 계약은 clone만으로 확정할 수 없다.
NEXT READ	handshake에 generation과 last event identity를 넣기 전 server protocol 문서가 admitted되지 않았으므로 해당 claim은 UNKNOWN으로 유지한다.

다른 입력과 변경안에 전이해 볼 질문

1. session-not-found budget이 끝난 뒤 token refresh가 성공하면 어떤 state에서 새 시도를 허용할 것인가?
2. 재연결 뒤 첫 event가 이미 본 id라면 UI와 transcript는 각각 어떻게 deduplicate해야 하는가?

이 설명을 확인할 코드 src/remote/SessionsWebSocket.ts:208-300 src/remote/RemoteSessionManager.ts:106-183

SOURCE READING · 1/2

OBSERVED

SDK message adapter는 wire representation의 정보 손실을 명시한다

remote backend의 SDKMessage를 REPL Message로 바꿀 때 variant마다 보존되는 identity와 사라지는 metadata가 다르다.

SDK variant별 local conversion src/remote/sdkMessageAdapter.ts:24-118

```
24      * The CCR backend sends SDK-format messages via WebSocket. The REPL expects
25      * internal Message types for rendering. This adapter bridges the two.
26      */
27
28     /**
29      * Convert an SDKAssistantMessage to an AssistantMessage
30      */
31     function convertAssistantMessage(msg: SDKAssistantMessage): AssistantMessage {
32       return {
33         type: 'assistant',
34         message: msg.message,
35         uuid: msg.uuid,
36         requestId: undefined,
37         timestamp: new Date().toISOString(),
38         error: msg.error,
39       }
40     }
41
42     /**
43      * Convert an SDKPartialAssistantMessage (streaming) to a StreamEvent
44      */
45     function convertStreamEvent(msg: SDKPartialAssistantMessage): StreamEvent {
46       return {
47         type: 'stream_event',
48         event: msg.event,

… 65 lines omitted; use the source link for the complete range …

114       return {
115         type: 'system',
116         subtype: 'informational',
117         content: `Tool ${msg.tool_name} running for ${msg.elapsed_time_seconds}s…`,
118         level: 'info',
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	assistant는 message payload와 uuid를 보존하지만 local timestamp를 새로 만든다.
그 표현이 만든 조건	partial assistant는 StreamEvent로 바꾼다.
그 표현이 만든 조건	result/status/tool progress는 local SystemMessage로 축약한다.
그 표현이 만든 조건	tool progress는 필요한 tool-specific union이 없어 informational text가 된다.

언어와 런타임은 여기까지 보장한다

adapter는 구조를 복사하는 함수가 아니라 서로 다른 domain model 사이의 lossy projection이다. TypeScript switch가 variant coverage를 돕지만 output type이 더 좁으면 원래 정보를 모두 보존할 수 없다.

프로젝트가 이 메커니즘에 맡긴 책임

remote surface가 local component를 재사용하려면 wire schema를 canonical state로 직접 넣지 않고 이 anti-corruption layer를 지난다. 따라서 새로운 SDK variant는 adapter의 보존·축약·ignore 정책을 명시해야 한다.

경계를 잘못 읽을 때 생기는 위험

progress를 문자열로 축약하면 local UI의 tool-specific collapse와 resume가 달라질 수 있다. timestamp 재생성은 remote event time과 local observation time을 혼동하게 한다.

이 설명을 확인할 코드 src/remote/sdkMessageAdapter.ts:24-118

SOURCE READING · 1/2

OBSERVED

REPL bridge는 remote input을 command queue에 넣기 전에 identity와 command authority를 제한한다

inbound user message는 UUID를 보존해 중복 echo를 피하지만 slash command는 bridge-safe 판정과 skip flag를 함께 사용한다.

bridge lifetime과 duplicate guards

src/hooks/useReplBridge.tsx:45-105

```
45      *
46      * Silently skips if bridge is not enabled or user is not OAuth-authenticated.
47      *
48      * Watches AppState.replBridgeEnabled — when toggled off (via /config or footer),
49      * the bridge is torn down. When toggled back on, it re-initializes.
50      *
51      * Inbound messages from claude.ai are injected into the REPL via queuedCommands.
52      */
53     export function useReplBridge(messages: Message[], setMessages: (action: React.SetStateAction<Message[]>) => void,
abortControllerRef: React.RefObject<AbortController | null>, commands: readonly Command[], mainLoopModel: string): {
54       sendBridgeResult: () => void;
55     } {
56       const handleRef = useRef<ReplBridgeHandle | null>(null);
57       const teardownPromiseRef = useRef<Promise<void> | undefined>(undefined);
58       const lastWrittenIndexRef = useRef(0);
59       // Tracks UUIDs already flushed as initial messages. Persists across
60       // bridge reconnections so Bridge #2+ only sends new messages — sending
61       // duplicate UUIDs causes the server to kill the WebSocket.
62       const flushedUUIDsRef = useRef(new Set<string>());
63       const failureTimeoutRef = useRef<ReturnType<typeof setTimeout> | undefined>(undefined);
64       // Persists across effect re-runs (unlike the effect's local state). Reset
65       // only on successful init. Hits MAX_CONSECUTIVE_INIT_FAILURES → fuse blown
66       // for the session, regardless of replBridgeEnabled re-toggling.

… 34 lines omitted; use the source link for the complete range …

101           const outboundOnly = replBridgeOutboundOnly;
102           function notifyBridgeFailed(detail?: string): void {
103             if (outboundOnly) return;
104             addNotification({
105               key: 'bridge-failed',
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	flushedUUIDsRef는 reconnect를 넘어 이미 보낸 initial message identity를 기억한다.
그 표현이 만든 조건	consecutive failure ref는 session 단위 fuse 역할을 한다.
그 표현이 만든 조건	현재 command/model/messages는 ref로 최신 값을 callback에 제공한다.
그 표현이 만든 조건	feature gate는 dynamic bridge import를 external build에서 제거할 수 있게 한다.

언어와 런타임은 여기까지 보장한다

React effect의 cleanup/re-run과 network connection lifetime은 일치하지 않을 수 있다. ref는 reconnect generation을 넘는 identity와 최신 callback input을 보존하지만, durable journal이나 distributed dedup store는 아니다.

프로젝트가 이 메커니즘에 맡긴 책임

bridge는 UI adapter에만 머물지 않고 remote user input을 local command queue로 주입한다. UUID, attachment resolution, webhook sanitation, slash-command 제한은 remote transport가 local execution authority로 바뀌는 경계다.

경계를 잘못 읽을 때 생기는 위험

component remount나 process restart에서 ref dedup이 사라지고 server replay가 오면 같은 message가 재주입될 수 있다. skipSlashCommands 하나만 믿고 downstream override를 허용하면 remote input이 local immediate command를 실행할 수 있다.

이 설명을 확인할 코드 src/hooks/useReplBridge.tsx:45-105 src/hooks/useReplBridge.tsx:160-223

DISTINCTION TABLE

OBSERVED

Remote surface의 네 protocol을 분리한다

한 WebSocket 안에서도 data와 control, local projection, permission은 다른 identity와 terminal 규칙을 가진다.

Protocol	대표 값	보존해야 할 것
Data stream	SDK assistant/partial/status/result	message UUID·event order·terminal result
Control	interrupt·permission response·ack	request id·한 번의 결정
Connection	connect·reconnect·permanent close	generation·retry budget·gap
Local projection	Message·StreamEvent·queue command	loss/translation provenance

이 표를 읽는 관점

연결이 복구되었다는 사실만으로 data gap, 중복, 미결 permission request가 정산되었다고 볼 수 없다.

SELECTIVE EXECUTION TRACE

OBSERVED

remote prompt가 local effect를 시작하는 경계

bridge에서 들어온 prompt가 local turn으로 바뀌는 최소 경로를 따라간다.

책임 연결

Transport

WebSocket message를 runtime guard로 검증한다.

이때 남은 상태와 판단이 “Normalize”에서 사용할 전제가 된다.

책임 연결

Normalize

content·attachment·UUID를 local input 표현으로 바꾼다.

이때 남은 상태와 판단이 “Authority gate”에서 사용할 전제가 된다.

책임 연결

Authority gate

remote origin과 bridge-safe command policy를 붙인다.

이때 남은 상태와 판단이 “Queue”에서 사용할 전제가 된다.

책임 연결

Queue

local command queue에 identity를 보존해 enqueue한다.

이때 남은 상태와 판단이 “Turn”에서 사용할 전제가 된다.

책임 연결

Turn

REPL queue processor가 QueryGuard를 확보하고 일반 input pipeline을 실행한다.

이때 남은 상태와 판단이 “Projection”에서 사용할 전제가 된다.

관찰·정산

Projection

local query event를 bridge/SDK representation으로 다시 전송한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

remote origin은 local user input과 같은 text여도 provenance와 command capability를 끝까지 보존해야 한다.

어디에서 설명이 틀어지는가

normalization 뒤 origin tag가 사라지면 slash command와 permission prompt가 local interaction으로 잘못 승격될 수 있다.

이 설명을 확인할 코드 src/hooks/useReplBridge.tsx:160-223 src/screens/REPL.tsx:895-916

03 인증과 network transport는 request 바깥의 보안 상태 기계다

API call 한 줄 뒤에는 token source precedence, secure storage, refresh race, proxy·mTLS·CA selection이 있다. 이 경계를 생략하면 “인증 header를 붙인다”는 설명만 남고 secret lifetime과 corporate network behavior를 이해할 수 없다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 Auth source resolution은 mode와 provider에 따라 허용 credential universe를 줄인다	코드	진입 경계 OAuth refresh는 secure storage와 cross-process lock을 이용한 compare-after-lock protocol이다
코드	책임 연결 Proxy·CA·mTLS selection은 모든 URL에 같은 dispatcher를 붙이지 않는다	구분	책임 연결 Secret과 transport의 lifetime을 단계별로 나눈다

코드 독해의 시작점

`src/utils/auth.ts` → `src/utils/secureStorage/**` → `src/utils/proxy.ts` → `src/utils/caCerts.ts` → `src/services/api/client.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Auth source resolution은 mode와 provider에 따라 허용 credential universe를 줄인다

OAuth, env token, API key helper, file descriptor, keychain은 단순 fallback 목록이 아니라 bare·managed remote·third-party provider에서 서로 다른 허용 규칙을 가진다.

managed OAuth context와 token source resolution

src/utils/auth.ts:88-206

```
88      * who runs `claude` in their terminal with an API key sees every CCD session
89      * also use that key — and fail if it's stale/wrong-org.
90      */
91     function isManagedOAuthContext(): boolean {
92       return (
93         isEnvTruthy(process.env.CLAUDE_CODE_REMOTE) ||
94         process.env.CLAUDE_CODE_ENTRYPOINT === 'claude-desktop'
95       )
96     }
97
98     /** Whether we are supporting direct 1P auth. */
99     // this code is closely related to getAuthTokenSource
100    export function isAnthropicAuthEnabled(): boolean {
101      // --bare: API-key-only, never OAuth.
102      if (isBareMode()) return false
103
104      // `claude ssh` remote: ANTHROPIC_UNIX_SOCKET tunnels API calls through a
105      // local auth-injecting proxy. The launcher sets CLAUDE_CODE_OAUTH_TOKEN as a
106      // placeholder iff the local side is a subscriber (so the remote includes the
107      // oauth-2025 beta header to match what the proxy will inject). The remote's
108      // ~/.claude settings (apiKeyHelper, settings.env.ANTHROPIC_API_KEY) MUST NOT
109      // flip this — they'd cause a header mismatch with the proxy and a bogus
110      // "invalid x-api-key" from the API. See src/ssh/sshAuthProxy.ts.
111      if (process.env.ANTHROPIC_UNIX_SOCKET) {
112        return !!process.env.CLAUDE_CODE_OAUTH_TOKEN
… 89 lines omitted; use the source link for the complete range …
202        return { source: 'claude.ai' as const, hasToken: true }
203      }
204
205      return { source: 'none' as const, hasToken: false }
206    }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	bare mode는 OAuth·keychain·credential file을 배제하고 API-key 계열만 허용한다.
그 표현이 만든 조건	remote/Desktop managed context는 사용자의 terminal API key로 조용히 fallback하지 않는다.
그 표현이 만든 조건	Bedrock·Vertex·Foundry와 external token/key는 direct Anthropic auth를 비활성화할 수 있다.
그 표현이 만든 조건	env, file descriptor, helper, secure storage source는 실제 secret을 읽기 전에도 source/hasToken을 구분한다.

언어와 런타임은 여기까지 보장한다

credential precedence는 편의 설정이 아니라 trust boundary다. child/managed session이 parent terminal의 key를 물려받으면 잘못된 organization·billing·authority로 request가 나간다. source detection과 secret retrieval을 분리하면 trust dialog 전에 helper command를 실행하지 않는 같은 안전 정책을 구현할 수 있다.

프로젝트가 이 메커니즘에 맡긴 책임

auth module이 effective source와 refresh/storage lifecycle을 소유하고 API client는 최종 provider credential을 사용한다. provider 선택과 auth source는 독립 축이지만 잘못 결합되면 header 형식이 충돌한다.

경계를 잘못 읽을 때 생기는 위험

env와 keychain에 서로 다른 credential이 있을 때 source provenance를 잃으면 401의 원인을 찾기 어렵다. managed OAuth guard를 제거하면 stale user key가 remote session을 오염시킬 수 있다.

이 설명을 확인할 코드 src/utils/auth.ts:88-206

SOURCE READING · 1/2

OBSERVED

OAuth refresh는 secure storage와 cross-process lock을 이용한 compare-after-lock protocol이다

여러 terminal이 같은 expired token을 볼 때 모두 refresh하면 새 token이 서로를 revoke하거나 keychain read가 폭증할 수 있어, cache invalidation·in-flight dedup·file lock·재검사를 결합한다.

401 dedup, async secure read, refresh lock과 재검사

src/utils/auth.ts:1338-1561

```
1338  // In-flight dedup: when N claude.ai proxy connectors hit 401 with the same
1339  // token simultaneously (common at startup — #20930), only one should clear
1340  // caches and re-read the keychain. Without this, each call's clearOAuthTokenCache()
1341  // nukes readInFlight in macOsKeychainStorage and triggers a fresh spawn —
1342  // sync spawns stacked to 800ms+ of blocked render frames.
1343  const pending401Handlers = new Map<string, Promise<boolean>>()
1344
1345  /**
1346   * Handle a 401 "OAuth token has expired" error from the API.
1347   *
1348   * This function forces a token refresh when the server says the token is expired,
1349   * even if our local expiration check disagrees (which can happen due to clock
1350   * issues when the token was issued).
1351   *
1352   * Safety: We compare the failed token with what's in keychain. If another tab
1353   * already refreshed (different token in keychain), we use that instead of
1354   * refreshing again. Concurrent calls with the same failedAccessToken are
1355   * deduplicated to a single keychain read.
1356   *
1357   * @param failedAccessToken - The access token that was rejected with 401
1358   * @returns true if we now have a valid token, false otherwise
1359   */
1360  export function handleOAuth401Error(
1361    failedAccessToken: string,
1362  ): Promise<boolean> {
… 194 lines omitted; use the source link for the complete range …
1557    } finally {
1558      logEvent('tengu_oauth_token_refresh_lock_releasing', {})
1559      await release()
1560      logEvent('tengu_oauth_token_refresh_lock_released', {})
1561    }
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	같은 failed token의 401 handler는 in-flight Map으로 한 promise에 합류한다.
그 표현이 만든 조건	cache를 비우고 secure storage에서 읽은 token이 이미 달라졌으면 다른 process의 refresh를 재사용한다.
그 표현이 만든 조건	일반 refresh check도 process 안에서는 pending promise로 dedup한다.
그 표현이 만든 조건	cross-process lock을 얻은 뒤 token expiry를 다시 읽어 검사한다.
그 표현이 만든 조건	refresh 성공·실패 어느 경로에서도 cache와 lock을 정산한다.

언어와 런타임은 여기까지 보장한다

이 패턴은 double-checked locking의 비동기·cross-process 변형이다. lock을 기다리는 동안 world가 바뀔 수 있으므로 lock 전 검사만 믿지 않고 획득 뒤 durable storage를 다시 읽는다. access token 자체를 log identity로 남기지 않으면서 같은 실패를 dedup할 안전한 fingerprint가 필요하다.

프로젝트가 이 메커니즘에 맡긴 책임

secure storage가 durable token owner이고 process cache는 성능 projection이다. refresh effect의 commit은 새 token을 저장한 시점이며, API retry는 그 뒤 cache를 다시 읽어야 한다.

경계를 잘못 읽을 때 생기는 위험

lock timeout·process crash·storage write 실패·clock skew는 token이 refresh됐는지 불명확한 상태를 만든다. error에서 현재 storage token을 다시 확인하지 않고 무조건 재로그인을 요구하면 다른 process의 성공을 놓친다.

이 설명을 확인할 코드 src/utils/auth.ts:1338-1561 src/utils/auth.ts:1189-1298

SOURCE READING · 1/2

OBSERVED

Proxy·CA·mTLS selection은 모든 URL에 같은 dispatcher를 붙이지 않는다

Anthropic API, MCP transport, WebSocket, telemetry는 서로 다른 endpoint이므로 unix auth proxy와 corporate proxy·NO_PROXY·custom CA를 목적에 맞게 scope해야 한다.

Anthropic-only unix socket과 proxy/TLS fetch options src/utils/proxy.ts:278-320

```
278    * Get fetch options for the Anthropic SDK with proxy and mTLS configuration
279    * Returns fetch options with appropriate dispatcher for proxy and/or mTLS
280    *
281    * @param opts.forAnthropicAPI - Enables ANTHROPIC_UNIX_SOCKET tunneling. This
282    *   env var is set by `claude ssh` on the remote CLI to route API calls through
283    *   an ssh -R forwarded unix socket to a local auth proxy. It MUST NOT leak
284    *   into non-Anthropic-API fetch paths (MCP HTTP/SSE transports, etc.) or those
285    *   requests get misrouted to api.anthropic.com. Only the Anthropic SDK client
286    *   should pass `true` here.
287    */
288   export function getProxyFetchOptions(opts?: { forAnthropicAPI?: boolean }): {
289     tls?: TLSConfig
290     dispatcher?: undici.Dispatcher
291     proxy?: string
292     unix?: string
293     keepalive?: false
294   } {
295     const base = keepAliveDisabled ? ({ keepalive: false } as const) : {}
296
297     // ANTHROPIC_UNIX_SOCKET tunnels through the `claude ssh` auth proxy, which
298     // hardcodes the upstream to the Anthropic API. Scope to the Anthropic API
299     // client so MCP/SSE/other callers don't get their requests misrouted.
300     if (opts?.forAnthropicAPI) {
… 15 lines omitted; use the source link for the complete range …
316
317     // Otherwise, use TLS options directly if available
318     return { ...base, ...getTLSFetchOptions() }
319   }
320
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	ANTHROPIC_UNIX_SOCKET은 forAnthropicAPI가 참인 SDK client에만 적용한다.
그 표현이 만든 조건	proxy가 있으면 Bun proxy option 또는 undici dispatcher를 선택한다.
그 표현이 만든 조건	proxy가 없으면 직접 TLS/mTLS option만 돌려준다.
그 표현이 만든 조건	CA loader는 custom CA가 default trust store를 대체한다는 Node semantics 때문에 base CA와 extra cert를 함께 구성한다.

언어와 런타임은 여기까지 보장한다

transport adapter는 URL만 전달하지 않고 DNS resolution 위치, proxy bypass, certificate trust, client certificate, socket ownership을 결정한다. custom ca를 하나 지정하면 default root가 자동 유지된다고 가정할 수 없으므로 명시적 합성이 필요하다.

프로젝트가 이 메커니즘에 맡긴 책임

proxy module이 HTTP client별 adapter를 제공하고 CA/mTLS module이 trust material을 공급한다. 각 caller는 자신의 endpoint class를 알려 Anthropic-only tunnel이 다른 remote로 새지 않게 해야 한다.

경계를 잘못 읽을 때 생기는 위험

NO_PROXY 해석과 DNS가 local/proxy 중 어디서 일어나는지가 다르면 SSRF·allowlist 판단도 달라진다. extra CA read 실패를 조용히 default trust 성공으로 오해하거나 unix tunnel을 MCP에 재사용하면 보안과 가용성이 모두 깨진다.

이 설명을 확인할 코드 src/utils/proxy.ts:278-320 src/utils/caCerts.ts:1-104

DISTINCTION TABLE

OBSERVED

Secret과 transport의 lifetime을 단계별로 나눈다

인증을 header 하나로 축약하지 않고 secret이 존재하는 위치와 갱신 책임을 추적한다.

단계	owner·representation	지켜야 할 경계
Source selection	mode/provider/env/settings의 source tag	secret retrieval 전 trust와 precedence 결정
Durable storage	keychain 또는 credentials backend	token·refresh token의 최소 노출과 atomic update
Process cache	memoized token, pending refresh promise	401·mtime·setting 변화에서 invalidation
Refresh	cross-process lock + compare-after-lock	다른 process의 성공 재사용, 중복 revoke 방지
Request transport	API client header + proxy/TLS dispatcher	endpoint별 tunnel·CA·mTLS scope
Observation	source/error class/latency event	token·path·sensitive payload를 log하지 않음

이 표를 읽는 관점

repository source로는 client-side credential flow를 관찰할 수 있지만 server-side token retention·revocation·organization policy의 실제 운영은 UNKNOWN이다.

04

IDE·browser·voice·update는 주변 기능이 아니라 별도 platform adapters다

agent loop가 같아도 입력·출력·설치·권한의 물리 경계가 달라진다. 이 기능들은 존재 symbol, runtime gate, OS dependency, 실제 exercised 상태를 구분해 설명해야 한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	효과·위임 Claude in Chrome은 native host와 dynamic MCP를 동시에 조립한다	코드	효과·위임 IDE integration은 detected endpoint를 dynamic MCP config로 투영한다
코드	책임 연결 Auto updater는 version policy와 설치 serialization을 분리한다	해석	진입 경계 Voice는 gated UI가 아니라 native audio와 subprocess fallback의 platform matrix다
구분	책임 연결 제품 통합은 activation evidence ladder로 판정한다		

코드 독해의 시작점

`src/utils/claudeInChrome/**` → `src/hooks/useIDEIntegration.tsx` → `src/services/voice.ts` → `src/utils/autoUpdater.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Claude in Chrome은 native host와 dynamic MCP를 동시에 조립한다

browser extension integration은 tool 몇 개를 추가하는 것이 아니라 wrapper/manifest 설치, stdio MCP process, allowlisted tool name, system prompt를 한 activation bundle로 만든다.

auto-enable gate와 MCP/native-host composition src/utils/claudeInChrome/setup.ts:70-164

```
70    let shouldAutoEnable: boolean | undefined = undefined
71
72    export function shouldAutoEnableClaudeInChrome(): boolean {
73      if (shouldAutoEnable !== undefined) {
74        return shouldAutoEnable
75      }
76
77      shouldAutoEnable =
78        getIsInteractive() &&
79        isChromeExtensionInstalled_CACHED_MAY_BE_STALE() &&
80        (process.env.USER_TYPE === 'ant' ||
81          getFeatureValue_CACHED_MAY_BE_STALE('tengu_chrome_auto_enable', false))
82
83      return shouldAutoEnable
84    }
85
86    /**
87     * Setup Claude in Chrome MCP server and tools
88     *
89     * @returns MCP config and allowed tools, or throws an error if platform is unsupported
90     */
91    export function setupClaudeInChrome(): {
92      mcpConfig: Record<string, ScopedMcpServerConfig>
93      allowedTools: string[]
94      systemPrompt: string
… 65 lines omitted; use the source link for the complete range …
160           scope: 'dynamic' as const,
161           ...(hasEnv && { env }),
162         },
163       }
164
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	interactive session·extension detection·feature value가 auto-enable 후보를 만든다.
그 표현이 만든 조건	native build와 script build가 서로 다른 wrapper command를 생성한다.
그 표현이 만든 조건	MCP config는 현재 binary의 별도 entrypoint를 stdio server로 실행한다.
그 표현이 만든 조건	permission bypass mode는 child MCP environment에 명시적으로 전달된다.
그 표현이 만든 조건	allowedTools와 browser-specific system prompt가 함께 반환된다.

언어와 런타임은 여기까지 보장한다

native messaging manifest는 browser가 어느 executable을 실행해도 되는지 정하고, MCP config는 CLI가 그 process와 어떤 protocol로 통신할지 정한다. 둘 중 하나만 성공하면 integration은 symbol 수준에는 존재하지만 operational하지 않다.

프로젝트가 이 메커니즘에 맡긴 책임

setup 함수는 composition을 반환하면서 manifest 설치를 best-effort background effect로 시작한다. 따라서 returned MCP config와 실제 browser host 준비 완료는 같은 시점의 truth가 아니다.

경계를 잘못 읽을 때 생기는 위험

manifest install 실패를 swallow하면 current turn에 도구가 보이지만 extension이 연결되지 않는 부분 활성 상태가 생길 수 있다. permission mode를 child env에 전달할 때 local authority와 동일하다고 가정해서는 안 된다.

이 설명을 확인할 코드 src/utils/claudeInChrome/setup.ts:70-164 src/utils/claudeInChrome/setup.ts:165-220

SOURCE READING · 1/2

OBSERVED

IDE integration은 detected endpoint를 dynamic MCP config로 투영한다

React hook은 auto-connect policy를 평가한 뒤 IDE URL scheme에 따라 ws-ide 또는 sse-ide adapter를 app state에 한 번 추가한다.

IDE detection, auto-connect gate와 dynamic MCP projection src/hooks/useIDEIntegration.tsx:20-69

```
20        setDynamicMcpConfig,
21        setShowIdeOnboarding,
22        setIDEInstallationState
23      } = t0;
24      let t1;
25      let t2;
26      if ($[0] !== autoConnectIdeFlag || $[1] !== ideToInstallExtension || $[2] !== setDynamicMcpConfig || $[3] !==
setIDEInstallationState || $[4] !== setShowIdeOnboarding) {
27        t1 = () => {
28          const addIde = function addIde(ide) {
29            if (!ide) {
30              return;
31            }
32            const globalConfig = getGlobalConfig();
33            const autoConnectEnabled = (globalConfig.autoConnectIde || autoConnectIdeFlag || isSupportedTerminal() ||
process.env.CLAUDE_CODE_SSE_PORT || ideToInstallExtension || isEnvTruthy(process.env.CLAUDE_CODE_AUTO_CONNECT_IDE)) &&
!isEnvDefinedFalsy(process.env.CLAUDE_CODE_AUTO_CONNECT_IDE);
34            if (!autoConnectEnabled) {
35              return;
36            }
37            setDynamicMcpConfig(prev => {
38              if (prev?.ide) {
39                return prev;
40              }
41              return {
42                ...prev,
43                ide: {
44                  type: ide.url.startsWith("ws:") ? "ws-ide" : "sse-ide",
… 20 lines omitted; use the source link for the complete range …
65        t1 = $[5];
66        t2 = $[6];
67      }
68      useEffect(t1, t2);
69    }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	global setting, CLI flag, supported terminal, SSE port, install intent, env flag가 auto-connect 후보를 만든다.
그 표현이 만든 조건	명시적 falsy env는 다른 positive signal을 다시 막는다.
그 표현이 만든 조건	이미 ide config가 있으면 새 detection으로 덮지 않는다.
그 표현이 만든 조건	URL prefix가 transport type을 결정하고 auth token·platform metadata를 함께 보존한다.

언어와 런타임은 여기까지 보장한다

이 hook은 React lifecycle을 사용하지만 실제로 만드는 것은 외부 IDE server에 대한 dynamic transport configuration이다. effect 재실행과 IDE process lifetime이 다르므로 idempotent add와 stale endpoint disposition이 중요하다.

프로젝트가 이 메커니즘에 맡긴 책임

IDE detector가 endpoint truth를 소유하고 AppState의 dynamic MCP config는 현재 session projection을 소유한다. UI onboarding과 installation status는 연결 자체와 별도 state다.

경계를 잘못 읽을 때 생기는 위험

첫 endpoint가 죽은 뒤 prev.ide를 계속 보존하면 새 endpoint를 발견해도 교체하지 못한다. auth token을 UI/log로 흘리거나 ws/sse scheme을 잘못 분류하면 연결과 secret boundary가 깨진다.

이 설명을 확인할 코드 src/hooks/useIDEIntegration.tsx:20-69

SOURCE READING · 1/2

OBSERVED

Auto updater는 version policy와 설치 serialization을 분리한다

minimum/maximum version policy, registry discovery, global install, stale-lock recovery는 서로 다른 실패를 가지며 하나의 update 버튼으로 축약할 수 없다.

minimum/maximum version control policy src/utils/autoUpdater.ts:70-143

```
70    export async function assertMinVersion(): Promise<void> {
71      if (process.env.NODE_ENV === 'test') {
72        return
73      }
74
75      try {
76        const versionConfig = await getDynamicConfig_BLOCKS_ON_INIT<{
77          minVersion: string
78        }>('tengu_version_config', { minVersion: '0.0.0' })
79
80        if (
81          versionConfig.minVersion &&
82          lt(MACRO.VERSION, versionConfig.minVersion)
83        ) {
84          // biome-ignore lint/suspicious/noConsole:: intentional console output
85          console.error(`
86    It looks like your version of Claude Code (${MACRO.VERSION}) needs an update.
87    A newer version (${versionConfig.minVersion} or higher) is required to continue.
88
89    To update, please run:
90        claude update
91
92    This will ensure you have access to the latest features and improvements.
93    `)
94          gracefulShutdownSync(1)
… 44 lines omitted; use the source link for the complete range …
139
140   /**
141    * Checks if a target version should be skipped due to user's minimumVersion setting.
142    * This is used when switching to stable channel - the user can choose to stay on their
143    * current version until stable catches up, preventing downgrades.
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	minimum version은 dynamic config와 semver로 계속 실행 가능한지를 판단한다.
그 표현이 만든 조건	maximum version은 incident 중 auto-update 상한을 제공한다.
그 표현이 만든 조건	latest 조회는 project .npmrc 영향을 피하려고 home cwd에서 실행한다.
그 표현이 만든 조건	install은 O_EXCL lock file과 pid ownership으로 process 간 중복을 막는다.
그 표현이 만든 조건	finally가 성공·실패 모두 lock release를 시도한다.

언어와 런타임은 여기까지 보장한다

version selection은 control-plane policy이고 package installation은 local filesystem/process effect다. SemVer build metadata를 compatibility에서는 무시하지만 update detection에서는 exact string으로 비교하는처럼 목적별 equality도 다르다.

프로젝트가 이 메커니즘에 맡긴 책임

update lock은 한 process만 install effect를 소유하게 하지만 package manager 자체의 atomicity나 downloaded artifact authenticity를 이 코드만으로 증명하지 않는다.

경계를 잘못 읽을 때 생기는 위험

stale lock을 unlink하기 직전 재검사하지 않으면 두 process가 동시에 새 lock을 지울 수 있다. project cwd에서 npm view/install을 실행하면 악성 local config가 registry를 바꿀 수 있다.

이 설명을 확인할 코드 src/utils/autoUpdater.ts:70-143 src/utils/autoUpdater.ts:176-275 src/utils/autoUpdater.ts:319-350 src/utils/autoUpdater.ts:456-532

CONCEPT IN CONTEXT

OBSERVED

Voice는 gated UI가 아니라 native audio와 subprocess fallback의 platform matrix다

voice mode를 “음성을 text로 바꾼다”로만 설명하면 first-use dynamic import, OS permission, audio device probe, native/arecord/SoX fallback과 remote 환경 exclusion을 놓친다.

코드에서 읽어낼 수 있는 단서

출발 단서	native audio module은 첫 voice keypress에 lazy import되어 startup blocking을 피한다.
연결되는 단서	Linux에서는 device 존재와 arecord probe를 분리해 binary 존재만으로 availability를 확정하지 않는다.
연결되는 단서	macOS microphone permission은 실제 짧은 recording probe로 확인한다.
연결되는 단서	remote environment에는 local microphone이 없으므로 기능 존재와 현재 사용 가능성이 갈린다.

이 단서들을 하나의 설계로 묶으면

platform adapter의 핵심은 같은 high-level feature가 OS마다 다른 executable·permission·latency·cleanup contract를 가진다는 점이다. activation evidence는 source 존재에서 끝나지 않고 gate, dependency, permission, device open, capture, transcription까지 단계적으로 올라간다.

시스템 이해에 남는 결론

이 snapshot에서 voice source와 gate는 관찰되지만 실제 hardware·OS permission·production flag 값과 transcription backend 운영은 UNKNOWN이다.

이 설명을 확인할 코드 src/services/voice.ts:1-83 src/services/voice.ts:213-275

DISTINCTION TABLE

OBSERVED

제품 통합은 activation evidence ladder로 판정한다

통합 기능을 발견했다고 곧 사용할 수 있다고 표현하지 않는다.

단계	필요한 evidence	허용되는 표현
Symbol	type/import/feature name	통합을 참조하는 code가 존재한다
Included	현재 build에 implementation과 entrypoint 포함	이 variant가 기능을 구성할 수 있다
Configured	extension/endpoint/device/channel과 effective gate	현재 process가 활성화를 시도한다
Connected	handshake·device open·native host/MCP ready	local adapter가 외부 surface와 연결됐다
Exercised	tool call/audio/update result와 terminal receipt	특정 실행이 관찰됐다
Operational	배포 fleet·backend·rollout observation	이 저장소만으로는 대체로 UNKNOWN

이 표를 읽는 관점

Chrome·IDE·voice·update는 서로 다른 ladder를 가지지만 “symbol→included→configured→exercised”를 건너뛰지 않는 원칙은 같다.

05 제품 변형과 durable configuration

compile-time gate, runtime flag, settings source, migration은 서로 다른 시간축의 변화다.

이 장의 논증은 다음 질문을 차례로 해결한다

구분	책임 연결 변형 메커니즘의 시간축	알고리즘	기록·복구 Settings precedence resolution
알고리즘	기록·복구 Remote managed settings checksum과 polling	알고리즘	기록·복구 Idempotent migration
코드	진입 경계 Build-time gate와 runtime flag를 한 표에 넣되 같은 것으로 부르지 않는다	구분	책임 연결 Activation evidence ladder

코드 독해의 시작점

`bun:bundle` → `src/utils/settings/**` → `src/migrations/**` → `src/services/remoteManagedSettings/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

DISTINCTION TABLE

OBSERVED

변형 메커니즘의 시간축

언제 값이 결정되는지가 rollback과 관찰 가능성을 바꾼다.

메커니즘	결정 시점	변경 후
feature()/MACRO	build	새 bundle 필요
env/CLI	process start	재시작 필요
settings	load/refresh	scope와 precedence에 따라
GrowthBook	runtime cache/refresh	원격 값과 stale 가능
migration	durable state read/write	과거 표현 변경

이 표를 읽는 관점

runtime if처럼 보이는 feature()는 build-time intrinsic일 수 있다.

ALGORITHM A34 · 독자의 질문

Settings precedence resolution 은 어떤 시스템 문제를 닫는가

user·project·local·flag·managed source가 같은 setting key를 제공하면 effective value뿐 아니라 어느 source가 이겼는지 알아야 정책과 customization을 설명할 수 있다.

코드가 필요해지는 이유

user, project, local, flag, managed policy에서 온 값을 effective configuration으로 만든다.

OBSERVED settings constants와 resolver가 source order, effective value, winning source를 결정한다.

INFERENCE 각 source가 원본 설정을 소유하고 resolver가 precedence projection을 소유한다.

이 절에서 답할 결정 질문

ordered source list가 nested value와 provenance를 잃지 않고 effective configuration을 만드는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **layered configuration과 precedence.** 나중 layer가 앞 값을 덮는 간단한 모델도 nested object에서는 replace와 deep merge의 의미가 다르다. winner만 남기면 rejected candidate를 설명할 수 없다.
>
> 3–22행의 source 순서를 실제 merge site와 연결해 읽는다.

이 설명을 확인할 코드 src/utils/settings/constants.ts:3-22

ALGORITHM A34 · 실제 코드

OBSERVED

Settings precedence resolution 의 결정점은 어디에 있는가

값 병합은 source authority와 provenance를 함께 다뤄야 한다.

입력 표현. source별 SettingsJson

보존되는 작업 상태. ordered source list, 그리고 effective value와 source

ordered setting sources

src/utils/settings/constants.ts:3-22

```
3     /**
4      * All possible sources where settings can come from
5      * Order matters - later sources override earlier ones
6      */
7     export const SETTING_SOURCES = [
8       // User settings (global)
9       'userSettings',
10
11      // Project settings (shared per-directory)
12      'projectSettings',
13
14      // Local settings (gitignored)
15      'localSettings',
16
… 3 lines omitted; use the source link for the complete range …
20      // Policy settings (managed-settings.json or remote settings from API)
21      'policySettings',
22    ] as const
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A34 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

source별 SettingsJson은 처음부터 신뢰된 실행 상태가 아니다. ordered source list, 그리고 effective value와 source가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 활성 source를 선택한다. 이 결과가 다음 판단 "정해진 순서로 값을 병합한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 정해진 순서로 값을 병합한다. 이 결과가 다음 판단 "flag와 policy를 항상 포함한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** flag와 policy를 항상 포함한다. 이 결과가 다음 판단 "editable/read-only source를 구분한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** editable/read-only source를 구분한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

later override 규칙이 일관된다. policy/flag는 UI edit 대상이 아니다. effective value의 source를 설명할 수 있다.

ALGORITHM A34 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** later override 규칙이 일관된다.
- **프로토콜·식별의 질문으로 읽기.** policy/flag는 UI edit 대상이 아니다.
- **권한·신뢰 경계의 질문으로 읽기.** effective value의 source를 설명할 수 있다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- object spread가 nested semantics를 잃음
- source provenance 소실
- remote cache stale

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 조직 policy와 local customization을 한 설정 모델에서 합성한다.

RECOMMENDATION 각 effective key에 winning source와 rejected candidates를 진단 projection으로 제공한다.

ALGORITHM A34 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

활성 source를 순서대로 읽어 effective SettingsJson을 만든다.

실패 뒤 남는 상태

shallow spread, provenance 소실, stale remote cache는 예상하지 못한 winner를 만든다.

OBSERVED	설정 source의 명시적 순서가 정의돼 있다.
CONDITIONAL	실제 merge 구현이 이 순서와 nested semantics를 일관되게 적용할 때 precedence가 성립한다.
UNKNOWN	배포별 managed/remote 값과 cache freshness는 확인되지 않는다.
NEXT READ	새 source 추가 시 constants뿐 아니라 merge, diagnostics, sandbox projection을 함께 검증해야 한다.

다시 적용할 질문

1. managed source가 nested object 한 key만 주면 user object의 나머지를 보존할 것인가 교체할 것인가?
2. effective 값은 같지만 winning source가 바뀌면 어떤 운영 의미가 달라지는가?

ALGORITHM A35 · 독자의 질문

Remote managed settings checksum과 polling은 어떤 시스템 문제를 닫는가

조직 설정을 자주 fetch하면 비용이 크고 오래 cache하면 폐기된 policy를 쓸 수 있다. checksum과 polling은 fetched·cached·applied 상태를 구분해야 한다.

코드가 필요해지는 이유

조직 설정을 원격에서 받아 local effective settings에 반영하되 불필요한 write와 반복 fetch를 줄인다.

OBSERVED — remoteManagedSettings가 loading promise, checksum/cache, poll timer와 eligibility를 소유한다.

INFERENCE — remote control plane이 desired policy를 제공하고 local service가 fetch/cache/apply lifecycle을 소유한다.

이 절에서 답할 결정 질문

같은 checksum을 skip하는 최적화가 freshness와 partial write의 상태를 어떻게 표현하는가?

이 설명을 확인할 코드 src/services/remoteManagedSettings/index.ts:105-155 src/services/remoteManagedSettings/index.ts:584-638

ALGORITHM A35 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/remoteManagedSettings/index.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

stale-while-revalidate와 canonical hashing

cache를 즉시 사용하며 background에서 새 값을 확인한다. 동일 의미의 JSON은 canonical hash가 같아야 하지만 key order나 부분 write가 hash 의미를 왜곡할 수 있다.

> **실제 코드로 돌아가면.** 105–155행에서 eligibility, loading dedup, checksum 비교, timer 설치 순서를 본다.

이론은 정답이 아니라 코드에 던질 질문이다

검증·관측·운영
`control plane · stale-while-revalidate`
최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

상태·계산 모델
`canonical hashing`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

ALGORITHM A35 · 실제 코드

OBSERVED

Remote managed settings checksum과 polling의 결정점은 어디에 있는가

원격 policy는 fetch·auth·checksum·disk cache·background poll을 가진 control plane이다.

입력 표현. endpoint/auth, remote SettingsJson, 그리고 poll interval

보존되는 작업 상태. loading promise, checksum/cache, 그리고 poll timer

checksum과 eligibility

src/services/remoteManagedSettings/index.ts:105-155

```
105  function getRemoteManagedSettingsEndpoint() {
106    return `${getOauthConfig().BASE_API_URL}/api/claude_code/settings`
107  }
108
109  /**
110   * Recursively sort all keys in an object to match Python's json.dumps(sort_keys=True)
111   */
112  function sortKeysDeep(obj: unknown): unknown {
113    if (Array.isArray(obj)) {
114      return obj.map(sortKeysDeep)
115    }
116    if (obj !== null && typeof obj === 'object') {
117      const sorted: Record<string, unknown> = {}
118      for (const key of Object.keys(obj).sort()) {
… 34 lines omitted; use the source link for the complete range …
153   * - Loading was never started
154   */
155  export async function waitForRemoteManagedSettingsToLoad(): Promise<void> {
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A35 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

endpoint/auth, remote SettingsJson, 그리고 poll interval은 처음부터 신뢰된 실행 상태가 아니다. loading promise, checksum/cache, 그리고 poll timer가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** eligibility와 auth를 확인한다. 이 결과가 다음 판단 "retry policy로 remote settings를 fetch한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** retry policy로 remote settings를 fetch한다. 이 결과가 다음 판단 "key order를 canonicalize해 checksum을 계산한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** key order를 canonicalize해 checksum을 계산한다. 이 결과가 다음 판단 "변경 시 안전 검사와 disk save를 수행한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 변경 시 안전 검사와 disk save를 수행한다. 이 결과가 다음 판단 "timer로 refresh를 반복한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** timer로 refresh를 반복한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

동일 settings는 동일 checksum을 낸다. policy source는 local edit와 구분된다. poll 실패 disposition을 availability/security 관점에서 명시한다.

ALGORITHM A35 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **검증·관측·운영의 질문으로 읽기.** 동일 settings는 동일 checksum을 낸다.
- **상태·계산 모델의 질문으로 읽기.** policy source는 local edit와 구분된다.
- **검증·관측·운영의 질문으로 읽기.** poll 실패 disposition을 availability/security 관점에서 명시한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- stale policy 사용
- 부분 write
- auth 변화
- reject 시 app behavior

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 중앙 policy 업데이트와 local cache availability를 결합한다.

RECOMMENDATION applied revision, fetched revision, stale age를 UI/audit에 노출한다.

ALGORITHM A35 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

eligible한 경우 fetch를 단일화하고 checksum이 바뀐 settings를 cache/apply한 뒤 poll을 예약한다.

effect가 시작되고 관찰되는 경계

local policy file/state가 바뀌며 이후 permission과 sandbox에 영향을 준다.

현재 설계가 선택한 trade-off와 비용

중앙 policy 업데이트와 local cache availability를 결합한다. 이 선택의 계산·운영 비용은 canonical sort/hash는 object 크기에 O(K log K) 근사, network 별도.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

stale cache, partial write, auth 변화, rejected fetch 뒤 fallback은 정책 적용 상태를 모호하게 한다. 이때 함께 대입할 반례는 stale policy 사용, 부분 write, auth 변화, 그리고 reject 시 app behavior이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

fetched revision, applied revision, checksum, stale age, failure disposition을 노출해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A35 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	eligibility, loading promise, checksum/cache, polling 경로가 있다.
CONDITIONAL	canonical hash와 write가 일관되고 poll timer가 단일화될 때 local state가 remote에 수렴한다.
UNKNOWN	실제 endpoint revision 계약과 outage 시 요구되는 fail policy는 확인되지 않는다.
NEXT READ	applied/fetched revision을 분리하기 전 remote API authority 문서가 없으므로 운영 보장은 UNKNOWN으로 남긴다.

다른 입력과 변경안에 전이해 볼 질문

1. fetch는 성공했지만 local write가 실패하면 checksum을 갱신해도 되는가?
2. auth identity가 바뀌면 이전 cache를 즉시 폐기할지 stale fallback으로 쓸지 누가 결정하는가?

이 설명을 확인할 코드 src/services/remoteManagedSettings/index.ts:105-155 src/services/remoteManagedSettings/index.ts:584-638

ALGORITHM A36 · 독자의 질문

Idempotent migration 은 어떤 시스템 문제를 닫는가

과거 persisted model setting을 영구히 여러 reader가 해석하게 두는 대신 startup migration으로 현재 canonical form에 수렴시킨다. 중간 실패와 재실행이 안전해야 한다.

코드가 필요해지는 이유

과거 버전의 persisted configuration을 현재 semantic value로 옮긴다.

OBSERVED migration이 legacy key/value를 확인하고 current value와 marker를 갱신한다.

INFERENCE startup migration이 persisted representation 변환을 소유하고 runtime consumer는 canonical form만 읽는다.

이 절에서 답할 결정 질문

marker와 value write의 순서가 왜 멱등성과 crash recovery를 결정하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **멱등 migration과 schema evolution.** 한 번 성공한 migration을 다시 실행해도 같은 결과여야 한다. marker를 먼저 쓰고 value write가 실패하면 다음 실행이 작업을 건너뛸 수 있다.
>
> 1–57행에서 eligibility, value update, marker 순서와 alias 판정을 본다.

이 설명을 확인할 코드 src/migrations/migrateLegacyOpusToCurrent.ts:1-57

ALGORITHM A36 · 실제 코드

OBSERVED

Idempotent migration 의 결정점은 어디에 있는가

durable 설정은 새 코드가 오래된 key와 model alias를 반복 실행 안전하게 변환해야 한다.

입력 표현. 현재 settings/state, 그리고 legacy key/value

보존되는 작업 상태. migration marker 또는 변환된 value

model setting migration

src/migrations/migrateLegacyOpusToCurrent.ts:1-57

```
1     import {
2       type AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
3       logEvent,
4     } from '../services/analytics/index.js'
5     import { saveGlobalConfig } from '../utils/config.js'
6     import { isLegacyModelRemapEnabled } from '../utils/model/model.js'
7     import { getAPIProvider } from '../utils/model/providers.js'
8     import {
9       getSettingsForSource,
10      updateSettingsForSource,
11    } from '../utils/settings/settings.js'
12
13    /**
14     * Migrate first-party users off explicit Opus 4.0/4.1 model strings.
… 40 lines omitted; use the source link for the complete range …
55          model as AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
56      })
57    }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A36 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

현재 settings/state, 그리고 legacy key/value은 처음부터 신뢰된 실행 상태가 아니다. migration marker 또는 변환된 value가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** legacy condition을 검사한다. 이 결과가 다음 판단 "이미 새 표현이면 no-op한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 이미 새 표현이면 no-op한다. 이 결과가 다음 판단 "관련 key를 일관되게 갱신한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 관련 key를 일관되게 갱신한다. 이 결과가 다음 판단 "persist 후 다음 startup에서 재실행해도 변화가 없어야 한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** persist 후 다음 startup에서 재실행해도 변화가 없어야 한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

반복 실행 결과가 같다. 사용자 명시 값과 default migration을 구분한다. 부분 실패 시 중간 상태를 탐지한다.

ALGORITHM A36 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **내구성·복구의 질문으로 읽기.** 반복 실행 결과가 같다.
- **내구성·복구의 질문으로 읽기.** 사용자 명시 값과 default migration을 구분한다.
- **내구성·복구의 질문으로 읽기.** 부분 실패 시 중간 상태를 탐지한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- marker 먼저 기록 후 value write 실패
- 새 alias를 legacy로 오인
- 여러 migration ordering 충돌

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 버전별 reader를 영구 유지하지 않고 startup에서 canonical form으로 수렴한다.

RECOMMENDATION migration id, before/after digest, atomic write와 rollback disposition을 기록한다.

ALGORITHM A36 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

legacy 상태를 감지해 current semantic value로 변환하고 완료 상태를 기록한다.

실패 뒤 남는 상태

marker-first partial commit, alias 오인, migration ordering conflict는 복구 불가능한 중간 상태를 만든다.

OBSERVED	legacy Opus setting을 current form으로 옮기는 migration이 있다.
CONDITIONAL	write 순서와 재실행 guard가 partial failure를 안전하게 처리할 때 멱등하다.
UNKNOWN	filesystem write atomicity와 다른 migration과의 전체 순서는 확정되지 않는다.
NEXT READ	atomic record/rollback을 도입하기 전 startup migration registry를 함께 대조해야 한다.

다시 적용할 질문

1. marker write 뒤 value write가 실패하면 다음 startup은 무엇을 보고 재시도하는가?
2. 새 model alias가 과거 legacy 값과 같을 때 migration이 오작동하지 않으려면 어떤 version이 필요한가?

SOURCE READING · 1/2

OBSERVED

Build-time gate와 runtime flag를 한 표에 넣되 같은 것으로 부르지 않는다

compile-time false는 code/module을 제거하고 runtime flag는 포함된 code의 branch를 고른다.

feature gate와 runtime config snapshot src/query.ts:280-296

```
280     const budgetTracker = feature('TOKEN_BUDGET') ? createBudgetTracker() : null
281
282     // task_budget.remaining tracking across compaction boundaries. Undefined
283     // until first compact fires — while context is uncompacted the server can
284     // see the full history and handles the countdown from {total} itself (see
285     // api/api/sampling/prompt/renderer.py:292). After a compact, the server sees
286     // only the summary and would under-count spend; remaining tells it the
287     // pre-compact final window that got summarized away. Cumulative across
288     // multiple compacts: each subtracts the final context at that compact's
289     // trigger point. Loop-local (not on State) to avoid touching the 7 continue
290     // sites.
291     let taskBudgetRemaining: number | undefined = undefined
292
293     // Snapshot immutable env/statsig/session state once at entry. See QueryConfig
294     // for what's included and why feature() gates are intentionally excluded.
295     const config = buildQueryConfig()
296
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	feature(TOKEN_BUDGET)는 compile-time variant다.
그 표현이 만든 조건	buildQueryConfig는 env/statsig/session snapshot이다.
그 표현이 만든 조건	feature gate는 config snapshot에서 의도적으로 제외된다.

언어와 런타임은 여기까지 보장한다

결정 시점이 다르면 test matrix와 rollback 방식도 다르다.

프로젝트가 이 메커니즘에 맡긴 책임

variant별 source coverage를 보고서에 표시해야 한다.

경계를 잘못 읽을 때 생기는 위험

외부 false snapshot만 보고 internal branch의 correctness를 평가할 수 없다.

이 설명을 확인할 코드 src/query.ts:280-296

DISTINCTION TABLE

OBSERVED

Activation evidence ladder

기능 존재 주장에 필요한 evidence 수준이다.

수준	evidence	허용 주장
1 Symbol	type/import/string	참조가 존재
2 Included	구현+reachable branch	variant에 포함 가능
3 Configured	effective flag/settings	현재 process가 선택
4 Exercised	runtime event/test	경로가 실행됨
5 Operational	배포 관찰/receipt	운영 활성

이 표를 읽는 관점

현재 복원 저장소는 대부분 1~2 수준만 제공한다.

06

Telemetry에서 리팩터링 oracle까지

운영 코드와 구조 개선을 연결하되 관찰 수치가 곧 품질 점수라고 오해하지 않는다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 Analytics fan-out과 sink isolation
구분	책임 연결 현재 구조의 리팩터링 우선순위
해석	책임 연결 프로젝트를 이해했다는 최종 기준
알고리즘	책임 연결 Batched exporter flush
사례	책임 연결 새 capability 도입의 전체 decision path

코드 독해의 시작점

`src/services/analytics/**` → `src/utils/sessionStorage.ts` → `src/query.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A37 · 독자의 질문

Analytics fan-out과 sink isolation은 어떤 시스템 문제를 닫는가

여러 subsystem이 공통 event를 내고 sink는 first-party나 Datadog 등으로 바뀐다. 한 sink의 지연이나 실패가 제품 동작과 다른 sink를 막지 않아야 한다.

코드가 필요해지는 이유

여러 subsystem이 공통 event를 기록하면서 first-party/Datadog 등 transport를 교체한다.

OBSERVED — analytics registry가 attached sinks와 sampling/kill state를 소유하고 logEvent가 tagged metadata를 fan-out한다.

INFERENCE — producer는 event 의미를, registry는 delivery 대상과 isolation을, sink는 transport를 소유한다.

이 절에서 답할 결정 질문

fan-out이 producer를 transport에서 분리하면서 schema·privacy·중복 전송을 어디에서 통제하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **observer fan-out과 data minimization.** 한 event를 여러 subscriber에 전달하되 느린 subscriber가 producer를 막지 않게 한다. 전송 가능하다는 사실과 수집해야 한다는 privacy 목적은 별도 판단이다.
>
> 72–165행에서 sink iteration, error isolation, sampling/kill 조건을 본다.

이 설명을 확인할 코드 src/services/analytics/index.ts:72-165

ALGORITHM A37 · 실제 코드

OBSERVED

Analytics fan-out과 sink isolation의 결정점은 어디에 있는가

event producer는 sink 목록에 기록하고 sink 실패를 product flow와 분리한다.

입력 표현. eventName, 그리고 typed/tagged metadata

보존되는 작업 상태. attached sinks, 그리고 sampling/kill state

analytics sink registry와 logEvent

src/services/analytics/index.ts:72-165

```
72    export type AnalyticsSink = {
73      logEvent: (eventName: string, metadata: LogEventMetadata) => void
74      logEventAsync: (
75        eventName: string,
76        metadata: LogEventMetadata,
77      ) => Promise<void>
78    }
79
80    // Event queue for events logged before sink is attached
81    const eventQueue: QueuedEvent[] = []
82
83    // Sink - initialized during app startup
84    let sink: AnalyticsSink | null = null
85
… 77 lines omitted; use the source link for the complete range …
163     await sink.logEventAsync(eventName, metadata)
164   }
165
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A37 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

eventName, 그리고 typed/tagged metadata은 처음부터 신뢰된 실행 상태가 아니다. attached sinks, 그리고 sampling/kill state가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** producer가 sanitized metadata로 logEvent를 호출한다. 이 결과가 다음 판단 "등록된 sink에 event를 fan-out한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 등록된 sink에 event를 fan-out한다. 이 결과가 다음 판단 "sync/async path를 구분한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** sync/async path를 구분한다. 이 결과가 다음 판단 "sink가 batching/retry/kill switch를 적용한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** sink가 batching/retry/kill switch를 적용한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

telemetry 실패가 core effect를 바꾸지 않는다. code/path/PII metadata 규칙을 지킨다. event name과 attempt/resource identity를 join할 수 있다.

ALGORITHM A37 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **검증·관측·운영의 질문으로 읽기.** telemetry 실패가 core effect를 바꾸지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** code/path/PII metadata 규칙을 지킨다.
- **검증·관측·운영의 질문으로 읽기.** event name과 attempt/resource identity를 join할 수 있다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- sink backpressure
- 민감 metadata
- 동일 event 중복
- effect와 outcome join 불가

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 관측 producer와 delivery backend를 분리한다.

RECOMMENDATION event schema와 retention/privacy purpose를 machine-readable catalog로 관리한다.

ALGORITHM A37 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

typed event metadata를 활성 sink에 fan-out하고 각 transport가 전송한다.

실패 뒤 남는 상태

backpressure, 민감 metadata, duplicate sink, effect/outcome join 부재는 관측을 왜곡한다.

OBSERVED	sink registry와 공통 logEvent fan-out이 있다.
CONDITIONAL	sink error가 격리되고 metadata가 허용 schema를 따를 때 producer가 transport와 분리된다.
UNKNOWN	실제 retention, backend join, privacy 운영은 source로 확인되지 않는다.
NEXT READ	schema catalog를 만들기 전 모든 producer metadata와 sink filtering을 inventory해야 한다.

다시 적용할 질문

1. 한 sink가 throw하면 다른 sink와 제품 operation은 계속되는가?
2. 같은 event가 두 sink에서 수집될 때 하나의 effect/outcome으로 join할 identity가 있는가?

ALGORITHM A38 · 독자의 질문

Batched exporter flush 은 어떤 시스템 문제를 닫는가

event마다 network call을 하면 비용이 크지만 buffer를 두면 process crash 전 데이터가 사라질 수 있다. batch size와 timer가 비용과 loss window를 정한다.

코드가 필요해지는 이유

각 event마다 network call을 하지 않고 process 종료 전 가능한 범위에서 전달한다.

OBSERVED Datadog exporter가 buffer, flush timer, in-flight request를 소유한다.

INFERENCE analytics fan-out 뒤의 한 transport adapter이며 제품 event 의미를 바꾸지 않아야 한다.

이 절에서 답할 결정 질문

buffer·timer·in-flight request가 중복 flush와 종료 손실을 어떻게 제한하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **batching과 amortized cost.** 여러 event가 한 request 비용을 나눠 가져 평균 overhead를 낮춘다. 그 대신 아직 flush되지 않은 buffer가 bounded loss window가 된다.
>
> 85–130행에서 size trigger, timer trigger, buffer swap, retry 중 상태를 본다.

이 설명을 확인할 코드 src/services/analytics/datadog.ts:85-130

ALGORITHM A38 · 실제 코드

OBSERVED

Batched exporter flush 의 결정점은 어디에 있는가

event를 buffer하고 size/time/shutdown 경계에서 전송해 latency와 신뢰성을 절충한다.

입력 표현. event records, batch size, 그리고 flush interval

보존되는 작업 상태. buffer, flush timer, 그리고 in-flight request

buffer flush scheduling

src/services/analytics/datadog.ts:85-130

```
85    function camelToSnakeCase(str: string): string {
86      return str.replace(/[A-Z]/g, letter => `_${letter.toLowerCase()}`)
87    }
88
89    type DatadogLog = {
90      ddsource: string
91      ddtags: string
92      message: string
93      service: string
94      hostname: string
95      [key: string]: unknown
96    }
97
98    let logBatch: DatadogLog[] = []
… 29 lines omitted; use the source link for the complete range …
128   }
129
130   export const initializeDatadog = memoize(async (): Promise<boolean> => {
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A38 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

event records, batch size, 그리고 flush interval은 처음부터 신뢰된 실행 상태가 아니다. buffer, flush timer, 그리고 in-flight request가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** event를 normalized record로 buffer한다. 이 결과가 다음 판단 "threshold 또는 timer에서 flush한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** threshold 또는 timer에서 flush한다. 이 결과가 다음 판단 "현재 batch를 분리해 전송한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 현재 batch를 분리해 전송한다. 이 결과가 다음 판단 "성공/실패 disposition을 기록하고 timer를 재설정한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 성공/실패 disposition을 기록하고 timer를 재설정한다. 이 결과가 다음 판단 "shutdown에서 final flush를 시도한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** shutdown에서 final flush를 시도한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

동시에 같은 record를 두 batch가 소유하지 않는다. timer가 process를 불필요하게 붙잡지 않는다. telemetry retry가 product retry를 방해하지 않는다.

ALGORITHM A38 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** 동시에 같은 record를 두 batch가 소유하지 않는다.
- **구성·변형·성능의 질문으로 읽기.** timer가 process를 불필요하게 붙잡지 않는다.
- **내구성·복구의 질문으로 읽기.** telemetry retry가 product retry를 방해하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- crash 전 buffer 손실
- 중복 전송
- retry 중 memory 성장

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED network overhead를 줄이면서 bounded loss window를 허용한다.

RECOMMENDATION delivery id와 bounded disk spool의 privacy tradeoff를 명시한다.

ALGORITHM A38 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

event를 buffer에 넣고 size 또는 interval에서 batch를 network로 보낸다.

실패 뒤 남는 상태

crash loss, duplicate delivery, retry buffer growth, 종료 전 미정산은 관측 공백을 만든다.

OBSERVED	buffer와 size/interval 기반 flush scheduling이 있다.
CONDITIONAL	timer와 in-flight guard가 중복 flush를 막고 process가 정상 종료할 때 bounded loss를 유지한다.
UNKNOWN	backend deduplication과 shutdown flush의 실제 보장은 확인되지 않는다.
NEXT READ	disk spool을 제안하기 전 privacy와 at-least-once 중복 비용을 함께 평가해야 한다.

다시 적용할 질문

1. timer flush와 size flush가 같은 tick에 겹치면 buffer를 누가 소유하는가?
2. network timeout 뒤 retry가 시작될 때 새 event를 같은 batch에 합칠 것인가 분리할 것인가?

DISTINCTION TABLE

RECOMMENDATION

현재 구조의 리팩터링 우선순위

라인 수가 아니라 semantic risk와 검증 가능성으로 순서를 정한다.

순서	대상	먼저 고정할 oracle
1	REPL/headless state ownership	동일 turn event/terminal
2	Tool contract 분리	tool contract characterization
3	Transcript replay	write→load round-trip
4	Permission decision trace	input digest와 precedence
5	Feature catalog	variant/activation evidence

이 표를 읽는 관점

검증 oracle이 없는 큰 이동은 마지막에 한다.

SELECTIVE EXECUTION TRACE

OBSERVED

새 capability 도입의 전체 decision path

MCP tool 하나를 추가한다고 가정한다.

책임 연결

Source

server/plugin provenance를 확인한다.

이때 남은 상태와 판단이 "Schema"에서 사용할 전제가 된다.

책임 연결

Schema

external schema를 runtime validator로 변환한다.

이때 남은 상태와 판단이 "Identity"에서 사용할 전제가 된다.

책임 연결

Identity

collision 없는 normalized name을 만든다.

이때 남은 상태와 판단이 "Policy"에서 사용할 전제가 된다.

책임 연결

Policy

effect class와 permission을 선언한다.

이때 남은 상태와 판단이 "Runtime"에서 사용할 전제가 된다.

진입 경계

Runtime

Tool registry와 query protocol에 합류한다.

이때 남은 상태와 판단이 "Durability"에서 사용할 전제가 된다.

기록·복구

Durability

result metadata와 transcript projection을 정의한다.

이때 남은 상태와 판단이 "Operations"에서 사용할 전제가 된다.

관찰·정산

Operations

event·kill switch·migration 필요성을 결정한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

discovery가 곧 authority나 activation을 부여하지 않는다.

어디에서 설명이 틀어지는가

한 단계라도 unknown이면 보고서에는 unknown으로 남긴다.

CONCEPT IN CONTEXT

INFERENCE

프로젝트를 이해했다는 최종 기준

새 요구의 owner, state lifetime, invariant, effect, projection, recovery를 예측할 수 있어야 한다.

코드에서 읽어낼 수 있는 단서

출발 단서	새 SDK event는 query, QueryEngine projection, schema, remote, REPL compatibility를 건드린다.
연결되는 단서	새 Edit version token은 Read observation, permission preview, commit check, transcript를 건드린다.
연결되는 단서	새 feature gate는 internal/external build matrix와 missing module 상태를 바꾼다.

이 단서들을 하나의 설계로 묶으면

파일 위치 암기보다 변경 영향 예측이 구조 이해를 증명한다.

시스템 이해에 남는 결론

이 예측을 source evidence와 executable invariant로 연결할 때 리팩터링 제안이 된다.

이 설명을 확인할 코드 src/QueryEngine.ts:175-213 src/Tool.ts:362-405

THEORY ATLAS · 1/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

상태·계산 모델

3개 알고리즘 · 4개 표현

입력이 어떤 중간 표현과 상태 전이를 거쳐 결과가 되는지, 그리고 각 전이 전후에 무엇이 계속 참이어야 하는지를 설명한다.

이 권에서 나타나는 방식. A32 MCP server name normalization과 A44 WebSocket close code별 bounded reconnect에서 이 관점이 결정적인 이유는 “결과는 API name pattern을 만족한다. 또한 permanent close는 reconnect하지 않는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. canonicalization(A32), pure function(A32), state machine(A44), canonical hashing(A35)

스터디 질문. 단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의

3개 알고리즘 · 4개 표현

여러 실행이 같은 상태를 읽고 쓰는 동안 순서, 취소, 재시도, 관찰 시점이 결과의 의미를 바꾸지 않게 하는 조건을 다룬다.

이 권에서 나타나는 방식. A33 Background plugin installation queue과 A44 WebSocket close code별 bounded reconnect에서 이 관점이 결정적인 이유는 “한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다. 또한 permanent close는 reconnect하지 않는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. background job(A33), eventual consistency(A33), retry budget(A44), batching(A38)

스터디 질문. stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

프로토콜·식별

2개 알고리즘 · 2개 표현

서로 다른 계층이 같은 작업과 같은 메시지를 가리킨다는 사실을 타입, 식별자, completion 규약으로 유지한다.

이 권에서 나타나는 방식. A32 MCP server name normalization과 A34 Settings precedence resolution에서 이 관점이 결정적인 이유는 “결과는 API name pattern을 만족한다. 또한 later override 규칙이 일관된다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. namespace collision(A32), precedence(A34)

스터디 질문. 생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 2/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

내구성·복구

2개 알고리즘 · 4개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A36 Idempotent migration과 A38 Batched exporter flush에서 이 관점이 결정적인 이유는 "반복 실행 결과가 같다. 또한 동시에 같은 record를 두 batch가 소유하지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. idempotence(A36), schema evolution(A36), forward migration(A36), at-least-once delivery(A38)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

권한·신뢰 경계

3개 알고리즘 · 3개 표현

실행 가능성, 허용 여부, 데이터 노출을 분리하고 신뢰하지 않은 입력이 effect 권한으로 곧바로 승격되지 않게 한다.

이 권에서 나타나는 방식. A33 Background plugin installation queue과 A34 Settings precedence resolution에서 이 관점이 결정적인 이유는 "한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다. 또한 later override 규칙이 일관된다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. supply-chain provenance(A33), provenance(A34), data minimization(A37)

스터디 질문. 누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능

3개 알고리즘 · 3개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A44 WebSocket close code별 bounded reconnect과 A34 Settings precedence resolution에서 이 관점이 결정적인 이유는 "permanent close는 reconnect하지 않는다. 또한 later override 규칙이 일관된다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. anti-corruption(A44), layered configuration(A34), amortized cost(A38)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 3/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

검증·관측·운영

2개 알고리즘 · 4개 표현

실행 결과를 관찰하고 회귀를 검출하며, cleanup과 운영 신호가 본래 동작을 왜곡하지 않도록 만드는 증거 체계를 다룬다.

이 권에서 나타나는 방식. A35 Remote managed settings checksum과 polling과 A37 Analytics fan-out과 sink isolation에서 이 관점이 결정적인 이유는 "동일 settings는 동일 checksum을 낸다. 또한 telemetry 실패가 core effect를 바꾸지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. control plane(A35), stale-while-revalidate(A35), observer/fan-out(A37), non-blocking telemetry(A37)

스터디 질문. 최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A32	**MCP server name normalization** 공백·점 등이 있는 server name을 tool namespace pattern에 맞춘다.	결과는 API name pattern을 만족한다.
A33	**Background plugin installation queue** 시작 시 필요한 plugin을 설치·갱신하고 진행 상태를 AppState에 반영한다.	한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다.
A44	**WebSocket close code별 bounded reconnect** remote session의 일시적 network 단절을 복구하면서 server가 확정한 종료나 존재하지 않는 session을 무한 재시도하지 않는다.	permanent close는 reconnect하지 않는다.
A34	**Settings precedence resolution** user, project, local, flag, managed policy에서 온 값을 effective configuration으로 만든다.	later override 규칙이 일관된다.
A35	**Remote managed settings checksum과 polling** 조직 설정을 원격에서 받아 local effective settings에 반영하되 불필요한 write와 반복 fetch를 줄인다.	동일 settings는 동일 checksum을 낸다.
A36	**Idempotent migration** 과거 버전의 persisted configuration을 현재 semantic value로 옮긴다.	반복 실행 결과가 같다.
A37	**Analytics fan-out과 sink isolation** 여러 subsystem이 공통 event를 기록하면서 first-party/Datadog 등 transport를 교체한다.	telemetry 실패가 core effect를 바꾸지 않는다.
A38	**Batched exporter flush** 각 event마다 network call을 하지 않고 process 종료 전 가능한 범위에서 전달한다.	동시에 같은 record를 두 batch가 소유하지 않는다.

SOURCE EVIDENCE INDEX · 1/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/services/mcp/normalization.ts:1-23	MCP server name normalization	algorithm
src/services/tools/toolExecution.ts:283-335	MCP capability가 Tool이 되기까지	walkthrough
src/services/plugins/PluginInstallationManager.ts:60-120	Background plugin installation queue	algorithm
src/screens/REPL.tsx:2701-2726	Skill은 코드보다 가볍지만 authority와 무관하지 않다	explain
src/remote/SessionsWebSocket.ts:208-300	WebSocket close code별 bounded reconnect	algorithm
src/remote/RemoteSessionManager.ts:106-183	WebSocket close code별 bounded reconnect	algorithm
src/remote/sdkMessageAdapter.ts:24-118	SDK message adapter는 wire representation의 정보 손실을 명시한다	code
src/hooks/useReplBridge.tsx:45-105	REPL bridge는 remote input을 command queue에 넣기 전에 identity와 command authority를 제한한다	code
src/hooks/useReplBridge.tsx:160-223	REPL bridge는 remote input을 command queue에 넣기 전에 identity와 command authority를 제한한다	code
src/screens/REPL.tsx:895-916	remote prompt가 local effect를 시작하는 경계	walkthrough
src/utils/auth.ts:88-206	Auth source resolution은 mode와 provider에 따라 허용 credential universe를 줄인다	code
src/utils/auth.ts:1338-1561	OAuth refresh는 secure storage와 cross-process lock을 이용한 compare-after-lock protocol이다	code
src/utils/auth.ts:1189-1298	OAuth refresh는 secure storage와 cross-process lock을 이용한 compare-after-lock protocol이다	code
src/utils/proxy.ts:278-320	Proxy·CA·mTLS selection은 모든 URL에 같은 dispatcher를 붙이지 않는다	code
src/utils/caCerts.ts:1-104	Proxy·CA·mTLS selection은 모든 URL에 같은 dispatcher를 붙이지 않는다	code
src/utils/claudeInChrome/setup.ts:70-164	Claude in Chrome은 native host와 dynamic MCP를 동시에 조립한다	code
src/utils/claudeInChrome/setup.ts:165-220	Claude in Chrome은 native host와 dynamic MCP를 동시에 조립한다	code
src/hooks/useIDEIntegration.tsx:20-69	IDE integration은 detected endpoint를 dynamic MCP config로 투영한다	code
src/utils/autoUpdater.ts:70-143	Auto updater는 version policy와 설치 serialization을 분리한다	code
src/utils/autoUpdater.ts:176-275	Auto updater는 version policy와 설치 serialization을 분리한다	code
src/utils/autoUpdater.ts:319-350	Auto updater는 version policy와 설치 serialization을 분리한다	code
src/utils/autoUpdater.ts:456-532	Auto updater는 version policy와 설치 serialization을 분리한다	code
src/services/voice.ts:1-83	Voice는 gated UI가 아니라 native audio와 subprocess fallback의 platform matrix다	explain
src/services/voice.ts:213-275	Voice는 gated UI가 아니라 native audio와 subprocess fallback의 platform matrix다	explain

SOURCE EVIDENCE INDEX · 2/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/utils/settings/constants.ts:3-22	Settings precedence resolution	algorithm
src/services/remoteManagedSettings/index.ts:105-155	Remote managed settings checksum과 polling	algorithm
src/services/remoteManagedSettings/index.ts:584-638	Remote managed settings checksum과 polling	algorithm
src/migrations/migrateLegacyOpusToCurrent.ts:1-57	Idempotent migration	algorithm
src/query.ts:280-296	Build-time gate와 runtime flag를 한 표에 넣되 같은 것으로 부르지 않는다	code
src/services/analytics/index.ts:72-165	Analytics fan-out과 sink isolation	algorithm
src/services/analytics/datadog.ts:85-130	Batched exporter flush	algorithm
src/QueryEngine.ts:175-213	프로젝트를 이해했다는 최종 기준	explain
src/Tool.ts:362-405	프로젝트를 이해했다는 최종 기준	explain

STUDY SYNTHESIS

확장과 운영 검증

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	MCP·plugin·skill의 trust boundary를 비교한다.
검증 질문	remote connection recovery와 message replay/duplication 보장을 구분한다.
검증 질문	wire SDK message가 local projection에서 무엇을 잃는지 기록한다.
검증 질문	feature와 runtime config의 결정 시점을 구분한다.
검증 질문	settings precedence와 migration idempotence를 사례로 검토한다.
검증 질문	새 capability의 activation evidence 수준을 판정한다.

이 권의 완료 기준

확장과 remote surface를 목록이 아니라 provenance·identity·authority·transport·durability·operations의 변화 계약으로 설명한다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.